

# DSCI 551 Final Report

**Name:** Bharath Prakash | **Email:** bharathp@usc.edu | **Team Size:** 1

**Title:** Disk-Backed B+Tree Index for Fast Lookups and Range Queries

## 1. Introduction & Motivation

The goal of this project is to implement a B+Tree index from scratch in Python and use SQLite as the reference production database system to empirically validate B+Tree performance against linear scans. B+Trees are the most widely used index structure in relational databases because they support both fast point lookups in  $O(\log n)$  time and efficient range scans through linked leaf nodes. This project addresses the core question: how much faster is an index than a full table scan, and does a hand-written Python B+Tree behave consistently with a production C-level implementation.

The motivation is rooted in course content: indexing is one of the most critical aspects of DBMS performance. Rather than studying B+Trees only theoretically, this project implements one from scratch and measures it against SQLite's native B+Tree index (written in C) on the same dataset. This creates a concrete, empirically grounded understanding of B+Tree internals, confirming that the performance advantages predicted by theory appear in real benchmark results.

The application uses a synthetic students table with 10,000 records and benchmarks three query methods: SQLite's native index scan, a hand-written Python B+Tree, and a forced full table scan baseline. All three operate on the same dataset, allowing direct performance comparison under controlled conditions.

## 2. Database System Overview

SQLite is the chosen database system for this project. It is a serverless, self-contained relational database engine that stores an entire database in a single cross-platform file. Unlike client-server databases such as PostgreSQL or MySQL, SQLite has no separate server process, the library runs in-process with the application. This architecture makes it one of the most widely deployed databases in the world, found in web browsers, mobile operating systems (Android and iOS), embedded systems, and desktop applications.

### Key Features

- Serverless, zero-configuration architecture requiring no installation or administration
- Single-file storage with full ACID transactions (atomicity, consistency, isolation, durability)
- Complete SQL support including JOINS, subqueries, triggers, views, and window functions
- Index-organized tables: row data lives directly in B-tree leaf nodes, not in a separate heap file

- Crash recovery via a write-ahead log (WAL) or rollback journal
- Default fixed page size of 4096 bytes, configurable up to 65536 bytes
- Widely embedded and used in Android, iOS, Firefox, Chrome, and over 1 trillion deployments worldwide

### Intended Use Cases

SQLite is optimized for embedded applications, local data storage, prototyping, mobile apps, and testing. It is not designed for high-concurrency multi-writer workloads, it uses database-level file locking, which serializes writes. Its sweet spot is single-writer, many-reader scenarios with moderate data sizes up to several gigabytes. For this project, SQLite's B+Tree index implementation and transparent EXPLAIN QUERY PLAN output made it the ideal system for studying how indexes behave internally.

## 3. Internal Architecture

SQLite's query execution pipeline consists of four major layers that work together to process every SQL statement. Understanding these layers is essential to understanding how B+Tree indexes affect application performance.

### 3.1 Execution Pipeline

- **SQL Compiler:** Parses SQL text, performs semantic analysis, and compiles the query into bytecode instructions for the Virtual Database Engine.
- **VDBE (Virtual Database Engine):** Executes bytecode opcode by opcode. Each SQL query becomes a small program of opcodes such as SeekRowid, Next, Column, and ResultRow. The VDBE is SQLite's query executor, equivalent to the execution engine in larger systems.
- **B-Tree Layer:** Manages how data is organized on disk. This is the core of SQLite's storage architecture. Both table data and index data are stored in B-trees, making the B-tree layer the most performance-critical component.
- **Pager:** Handles reading and writing fixed-size 4096-byte pages to and from disk. The pager manages transactions and crash recovery using either a rollback journal or write-ahead log.

### 3.2 Storage and Indexing

SQLite uses B-trees for both table storage and indexes, with a critical design difference from systems like PostgreSQL. Tables are index-organized: row data is stored directly in the leaf nodes of a B-tree keyed by rowid (or INTEGER PRIMARY KEY). PostgreSQL, by contrast, stores table data in a separate heap file and uses B-tree indexes only for lookups, requiring an additional I/O

hop to fetch the actual row. In SQLite, a primary key lookup is a single tree traversal with no secondary fetch required.

Additional storage details:

- Each B-tree node occupies exactly one 4096-byte page on disk
- Interior (internal) pages hold separator keys and child page pointers to guide traversal
- Leaf pages hold actual row data (for table B-trees) or (key, rowid) pairs (for index B-trees)
- For 10,000 rows with order approximately 100 keys per page, the tree is only 2-3 levels deep
- Point lookups therefore require just 2-3 page reads from disk
- Secondary indexes (e.g., on gpa) store (key, rowid) pairs, multiple rowids can share the same key for non-unique columns

### 3.3 Range Scans and the Linked Leaf Chain

The defining feature of B+Trees, as distinguished from plain B-trees, is the doubly-linked list connecting all leaf pages in sorted order. For a range scan (WHERE gpa BETWEEN 3.5 AND 3.8), SQLite traverses the idx\_gpa index B-tree to locate the first leaf where gpa >= 3.5, then follows linked leaf pages forward until gpa > 3.8. Each step reads one 4096-byte page sequentially, with no need to traverse back up to internal nodes. This linked-leaf design is what makes range scans cost O(k) in the number of matching rows rather than O(n) for the full table.

### 3.4 Query Execution Evidence

The EXPLAIN QUERY PLAN output from the application confirms exactly which path SQLite takes for each query type:

| Query Type                           | EXPLAIN QUERY PLAN Output                             | Access Path                  |
|--------------------------------------|-------------------------------------------------------|------------------------------|
| Point lookup (student_id = 5000)     | SEARCH students USING INTEGER PRIMARY KEY (rowid=?)   | Table B-tree, O(log n)       |
| Range scan (gpa BETWEEN 3.5 AND 3.8) | SEARCH students USING INDEX idx_gpa (gpa>? AND gpa<?) | Index B-tree + linked leaves |
| Forced scan (student_id + 0 = 5000)  | SCAN students                                         | Sequential full table scan   |

## 4. Application Design

The application is implemented as a Jupyter notebook (Bharath\_Prakash\_Implementation.ipynb) organized into two major parts: an SQLite benchmarking suite and a from-scratch Python B+Tree implementation. Both parts operate on the same 10,000-record dataset, enabling direct comparison.

### 4.1 SQLite Benchmarking (Part 1)

- Creates a SQLite database (students\_demo.db) with the students schema described below
- Creates a secondary index idx\_gpa on the gpa column
- Generates 10,000 synthetic student records programmatically using Python's sqlite3 module
- Benchmarks three query types: primary key index scan, secondary index range scan, and forced full table scan
- Uses EXPLAIN QUERY PLAN to verify which index path is taken for each query

### 4.2 Python B+Tree from Scratch (Part 2)

- Implements BPlusNode class: keys list, values list (child nodes or records), is\_leaf flag, and next/prev pointers for the linked leaf chain
- Implements BPlusTree class with configurable order (fanout), supporting insert(), search(), and range\_scan()
- Split logic handles leaf splits (promoting the first key of the new leaf) and internal node splits (promoting the middle key)
- Builds two B+Trees from the same SQLite data: one on student\_id (primary), one on gpa (secondary)
- Benchmarks the Python B+Tree against SQLite's index and forced linear scan using time.perf\_counter() with 1000 repetitions per query for sub-millisecond precision
- Verifies correctness: confirms that SQLite and B+Tree return identical result sets for both point lookups and range scans

### 4.3 Dataset Schema

| Field      | Description                                                                    |
|------------|--------------------------------------------------------------------------------|
| student_id | Integers 1 to 10,000 (sequential); INTEGER PRIMARY KEY (rowid alias)           |
| name       | Randomly generated first and last names (TEXT NOT NULL)                        |
| gpa        | Uniformly distributed floats in [2.0, 4.0] (REAL NOT NULL); indexed by idx_gpa |

## 4.4 Supported Operations

| Operation                  | Query Pattern             | Index Used             |
|----------------------------|---------------------------|------------------------|
| Point lookup               | WHERE student_id = ?      | Table B-tree (primary) |
| Range scan                 | WHERE gpa BETWEEN ? AND ? | Index B-tree (idx_gpa) |
| Full table scan (baseline) | WHERE student_id + 0 = ?  | None (forced scan)     |

## 5. Mapping Internals to Application Behavior

The internal B+Tree structure of SQLite directly determines the performance characteristics observed in the benchmarks. This section traces each observed result back to a specific architectural feature.

### 5.1 Point Lookup Performance

The benchmark results show a 7.39x speedup for SQLite's index scan over linear scan (0.1990 ms vs. 1.4698 ms, averaged over 100 queries x 1000 repetitions). This directly reflects the logarithmic depth of the B-tree. For 10,000 rows with order approximately 100 keys per page, the tree is only 2 levels deep. Each point lookup requires 2 page reads; the linear scan reads all data pages sequentially.

The Python B+Tree achieves a dramatically higher 245x speedup over linear scan (0.0060 ms) because it operates entirely in RAM with zero disk I/O. Tree traversal becomes pure Python list access rather than file system reads. The algorithm is identical to SQLite's, the difference is purely language-level and I/O overhead. This confirms that the B+Tree algorithm itself is the source of the speedup, not implementation-specific tricks.

### 5.2 Range Scan Performance

The linked leaf node design is the defining feature of B+Trees. SQLite's range scan on gpa navigates to the starting leaf via the index B-tree, then follows the doubly-linked leaf chain forward until the upper bound is exceeded. For each (gpa, rowid) pair, a second lookup fetches the full row from the table B-tree. The cost scales with the number of matching rows, not the table size.

The Python implementation replicates this exactly with the next pointer chain on BPlusNode leaf objects. The range\_scan() method traverses forward from the starting leaf using leaf.next until the high bound is exceeded. Correctness was verified: SQLite and the Python B+Tree return identical sorted result sets for all five tested GPA ranges (1,499 to 1,635 rows each).

### 5.3 Tree Height, Fanout, and Page Size

With order=100,  $\text{math.ceil}(\text{math.log}(10000, 100)) = 2$  levels. This matches SQLite's real tree depth. The high fanout (many keys per page) is a deliberate design: fitting approximately 100 keys

into a single 4096-byte page minimizes disk I/O, which is the primary bottleneck in on-disk databases. Fewer levels mean fewer page reads per lookup.

### 5.4 Index Build Cost

The Python B+Tree took 178 ms to build the `student_id` index and 147 ms for the `gpa` index (10,000 inserts each). This build cost reflects the write overhead inherent to B+Tree maintenance: each insert may trigger a leaf split, which in turn may trigger an internal node split propagating up to the root. In SQLite, this cost is paid incrementally at INSERT time.

### 5.5 Synthetic Data Design

GPA values were drawn from a uniform distribution over [2.0, 4.0] to ensure that each tested range query (e.g., 3.5 to 3.8, covering 15% of the range) returns approximately 15% of rows (roughly 1,500 rows). This makes range scan performance both meaningful and predictable, avoiding artificially skewed distributions that would make some ranges trivially fast.

## 6. Comparison with MySQL and MongoDB

This section compares SQLite against MySQL (InnoDB engine) and MongoDB (WiredTiger engine) along the dimensions most relevant to this project: indexing, storage, concurrency, and use cases.

### 6.1 SQLite vs. MySQL (InnoDB)

Both SQLite and MySQL InnoDB use B+Tree indexes as their primary index structure. However, there are important architectural differences in how secondary indexes work. MySQL InnoDB uses a clustered primary index (similar to SQLite's index-organized table) where the full row is stored in the primary key B-tree leaf. Secondary indexes in InnoDB store the primary key value rather than a physical rowid. A secondary index lookup therefore requires two B-tree traversals: one to find the primary key in the secondary index, then a second to fetch the row from the primary index. SQLite's secondary index stores the rowid directly, bypassing this double-lookup cost.

In terms of server architecture, MySQL requires a dedicated server process, supports row-level locking with MVCC (multi-version concurrency control), and is designed for multi-user concurrent write workloads which is standard for web applications. SQLite is serverless with file-level locking, making it unsuitable for high-concurrency writes but ideal for embedded and single-user scenarios.

### 6.2 SQLite vs. MongoDB (WiredTiger)

MongoDB is a document-oriented database using the WiredTiger storage engine, which implements a B-tree based structure with MVCC and optional compression. The data model is fundamentally different: MongoDB stores JSON/BSON documents with flexible, schema-less structure, while SQLite enforces a strict relational schema with typed columns.

Both systems support range queries on indexed fields. SQLite's range scans use the linked leaf structure described throughout this report. MongoDB range queries similarly traverse B-tree index nodes. MongoDB extends indexing with compound indexes, sparse indexes, TTL (time-to-live) indexes, and geospatial indexes, capabilities not available in SQLite.

MongoDB is designed for distributed, high-concurrency workloads with document-level locking, horizontal sharding, and replica sets. SQLite is single-process and single-writer. The appropriate choice depends entirely on the workload: structured relational data with single-writer access favors SQLite, variable document structures with high write concurrency and distribution favor MongoDB.

### 6.3 Summary Comparison Table

| Feature                | SQLite                | MySQL (InnoDB)        | MongoDB (WiredTiger)         |
|------------------------|-----------------------|-----------------------|------------------------------|
| Data model             | Relational            | Relational            | Document (JSON/BSON)         |
| Index structure        | B+Tree                | B+Tree (clustered)    | B-Tree (WiredTiger)          |
| Table storage          | Index-organized       | Clustered primary key | BSON documents               |
| Concurrency            | File-level lock       | Row-level lock (MVCC) | Document-level (MVCC)        |
| Server required        | No                    | Yes                   | Yes                          |
| Range scans            | Linked leaf traversal | Linked leaf traversal | B-tree traversal             |
| Secondary index stores | rowid                 | Primary key value     | Document <code>_id</code>    |
| Schema                 | Strict                | Strict                | Flexible                     |
| Best use case          | Embedded / local apps | Web apps, OLTP        | Semi-structured, distributed |

## 7. Limitations & Lessons Learned

### 7.1 Project Limitations

- **In-memory only.** The Python B+Tree does not persist nodes to disk. A true disk-backed implementation requires serializing nodes into fixed-size pages and managing a buffer pool, the original proposal goal, descope for a solo project.
- **No deletion support.** The tree supports insert, search, and range\_scan but not delete. Production indexes require node merging and redistribution to handle deletions.

- **Small dataset.** At 10,000 rows, query times are sub-millisecond and required 1000-repetition averaging for stability. A 1M+ row dataset would produce naturally stable differences.

## 7.2 Challenges Faced

- **Forcing index bypass.** SQLite's query planner aggressively uses indexes. Wrapping `student_id` in `student_id + 0 = ?` was the only way to force a full table scan, as SQLite exposes no SQL-level HINT or INDEX DISABLE option.
- **Range scan boundary bug.** The initial `range_scan()` missed records at the low boundary when a split left values in the preceding leaf. The fix required walking backward via `leaf.prev` before scanning forward.

## 7.3 Lessons Learned

- B+Tree height with high fanout (order=100) is just 2 levels for 10,000 rows, making  $O(\log n)$  concrete: 2 page reads vs. scanning hundreds of pages for a full table scan.
- The linked leaf chain is what separates B+Trees from B-trees, range scans need no backtracking through internal nodes once the starting leaf is found.

## 7.4 Future Work

- Implement disk persistence using fixed-size binary pages and a buffer pool to complete the originally proposed design.
- Add deletion with node merging and redistribution.
- Scale to 1 million rows for naturally stable benchmarks without repeated-run averaging.

## 8. Project Links

**GitHub Repository:** <https://github.com/bharathprakashusc/bplustree>

The GitHub repository contains jupyter notebook file of B+Tree implementation, README, and the generated dataset file.